

智能数学推理：形式化与定理证明

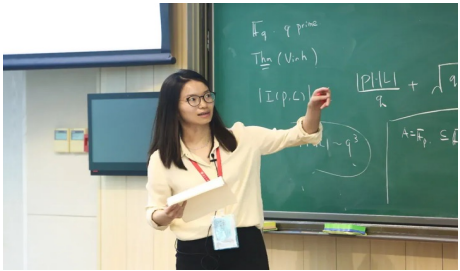
https://reaslab.io

文再文

北京大学
北京国际数学研究中心
国际机器学习研究中心
大数据分析与应用技术国家工程实验室

数学定理证明传统模式

- **数学是研究抽象结构与逻辑推理的基础学科，**为物理、信息、人工智能、工程、金融、生命科学等诸多领域提供理论语言与方法支撑
- **定理证明是数学的核心价值：**强调严谨证明奠定“可验证真理”标准，是数学区别于经验性学科的根本
- **尚未解决的著名猜想：**黎曼猜想、霍奇猜想、Navier–Stokes 方程光滑解存在性与光滑性、P≠NP ，哥德巴赫、孪生素数、Collatz 等等
- **几千年来基本未变的“纯手工”工作模式：**专家直觉+手工推理、演算与检查，“师傅带徒弟”的学徒制传承，很少受益于技术发展红利



王虹 挂谷（Kankeya）猜想



邓煜 希尔伯特第六问题

AI for Mathematics的重要性



美国NSF和Simons基金2025年资助卡内基梅隆大学成立计算机辅助数学推理（ICARM）研究所

聚焦计算机辅助数学推理与工具链建设，标志AI在数学研究中扮演越来越重要的角色，从根本上改变研究范式，推动AI驱动的数学任务和应用

AI+定理证明

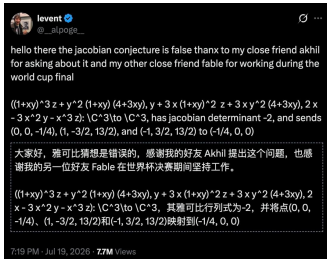
- **AI全自动定理证明**

  由计算机自动完成，基于逻辑推演
- **交互式定理证明**

  人机协同：专家提供策略，系统负责形式化检查
- **AI启发数学探索**

  大模型证明搜索与自然语言转形式化，能处理开放式自然语言问题、提升推理效率

Jacobian 猜想反例



支撑：数学形式化（Mathematical Formalization）

将数学概念、定理、证明用严格的形式语言表达，如 Lean、Coq、Isabelle 的语言，属于构建数学知识库和可机检证明的基础工程

DFP 收敛性反例

长期开放问题

一致凸目标 + Wolfe 线搜索，
经典 DFP 是否必然全局收敛?

$$f(x) = \frac{1}{2}\|x - C_\infty\|^2 + \sum_{k=0}^{\infty} \chi\left(\frac{x - x_k}{\rho_k}\right) a_k^T (x - x_k),$$

$$a_k = g_k - (x_k - C_\infty).$$

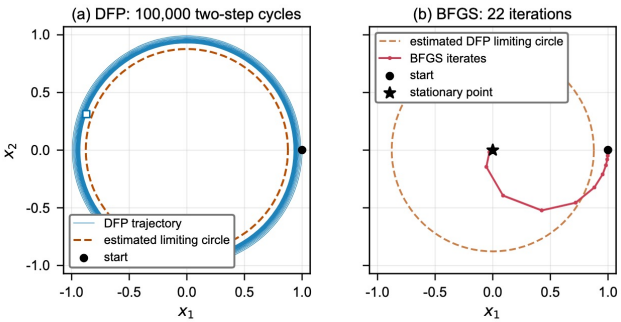
主要结论

构造 $f \in C^2(\mathbb{R}^2)$，满足

$$\tfrac{1}{2} I \preccurlyeq \nabla^2 f(x) \preccurlyeq 3/2\, I$$

所有步均满足标准强 Wolfe条件，
但 $\|\nabla f(x_k)\| \to G\infty > 0$

另有 $H_0 = I$ 的版本；结果推广到任意 $n \geq 2$。



DFP 轨迹逼近极限圆而不收敛；BFGS 对比快速到达驻点

结论：强 Wolfe 条件仍不足以保证 经典DFP算法 的全局收敛

可信推理
从探索到形式化验证

数学证明类别

- **自然语言(informal)证明**: 使用日常语言，可能结合数学符号来证明数学命题
- **形式化(formal)证明:** 使用严格数学和逻辑符号、机器可读语言表示数学证明，每一步明确可检查，不依赖于自然语言或直观推理
  - 有限句子（公理、假设、或者通过明确推理规则得出的）序列，通过明确的逻辑关系与其他句子连接起来
  - 命题逻辑、谓词逻辑、高阶逻辑
  - **依赖类型论 (Dependent Type Theory):** 结合了逻辑和类型，使得证明和程序之间的边界变得模糊。为构造复杂数学证明提供了强大的工具
- **交互式定理证明器（ITP)**
  - 用户与计算机交互，逐步构建证明，确保每一步都正确
  - 允许用户提取和验证证明对象，确保证明的完整性和正确性

梯度法形式化

- 考虑极小化可微函数 $f(x)$, 即 $\min_x f(x)$
- 固定步长梯度方法：$x_{k+1} = x_k - \alpha \nabla f(x_k)$

```
class GradientDescent_fix_stepsize (f : E → ℝ) (f' : E → E) (initial_x : E) :=
  (x : ℕ → E) (a : ℝ) (l : NNReal)
  (diff : ∀ x₁, HasGradientAt f (f' x₁) x₁) (smooth : LipschitzWith l f')
  (update : ∀ k : ℕ, x (k + 1) = x k - a • f' (x k))
  (hl : 1 > (0 : ℝ)) (step₁ : a > 0) (initial : x 0 = initial_x)
```

- 假设$f(x)$是凸函数，则有复杂度分析：

$$f(x_k) - f(x^*) \leq \frac{1}{2\alpha k}\|x_0 - x^*\|^2, \quad \forall k \in \mathbb{N}$$

```
theorem gradient_method_fix_stepsize {alg : GradientDescent_fix_stepsize f f' x₀}
    (hfun: ConvexOn ℝ Set.univ f) (step₂ : alg.a ≤ 1 / alg.l) :
    ∀ k : ℕ, f (alg.x (k + 1)) - f xm ≤ 1 / (2 * (k + 1) * alg.a) * ‖x₀ - xm‖ ^ 2 := by
```

数学优化和数值代数形式化

形式化定理库: Optlib and MatDecompFormal
https://github.com/optsuite/optlib
https://github.com/optsuite/MatDecompFormal

3000余条形式化定理 几万余行形式化代码

概念扩充

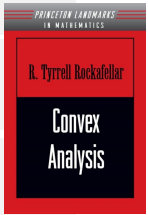
- ❑ 光滑函数梯度的定义及性质
- ❑ 非光滑函数次微分和近邻算子的定义

性质拓展

- ❑ 凸函数和Lipschitz光滑函数的性质
- ❑ 凸优化和一般约束优化问题的最优性条件

算法定义 理论证明

- ❑ 优化算法的收敛性分析
  - ❑ 梯度下降法　　次梯度算法
  - ❑ 近似点梯度算法　Nesterov加速算法
  - ❑ 交替方向乘子法　分块坐标下降法



部分形式化代码已经被官方Mathlib4采用

Chenyi Li, Ziyu Wang, Wanyi He, Yuxuan Wu, Shengyang Xu, and **Zaiwen Wen**. 2025. Formalization of Convergence Rates of Four First-order Algorithms. Journal of Automatic Reasoning. 69, 4 (Oct 2025)

开展六期数学形式化培训班
培养上千位高质量形式化人才



2025年7月 暑期培训班



2025年1月 寒假培训班

形式化智能体：在Lean反馈中闭环修正

形式化定理证明体

① **语义解析 目标识别**

识别数学定义、条件与目标

② **命题建模 陈述规范**

构造形式化命题与定理骨架

③ **定理检索 目标分解**

检索相关定理并拆解子目标

④ **证明生成 反馈修复**

生成 Lean 证明并根据反馈修复

⑤ **编译验证 成果沉淀**

闭环验证，沉淀可复用成果

Lean-MCP工具:

智能体·定理检索 ·编译验证

https://github.com/optsuite/lean-tools-mcp

告别低效人工形式化

输入材料：自然语言 | LaTex | Markdown | PDF | 图像 | Lean项目

输出结果：Lean代码 | 定理库 | 证明报告 | 错误日志

知识与工具

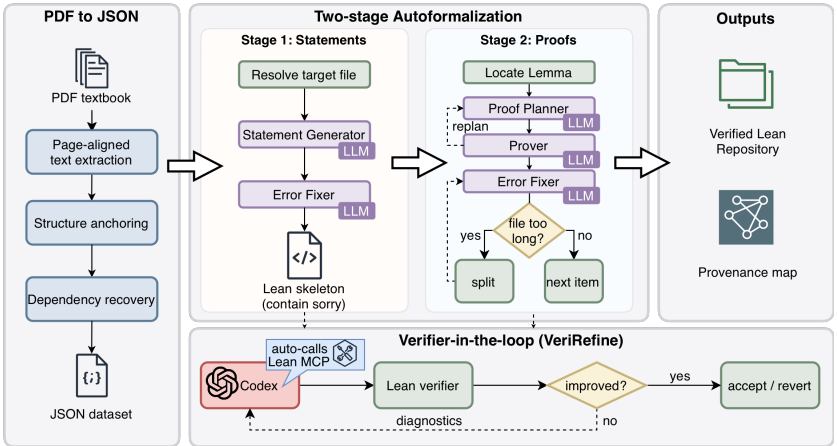
多源知识 + 交互验证

- 预加载知识库：Mathlib ReasLib
- 交互环境：Lean4 IDE Infoview
- 协作管理：Git同步 历史记录

核心特性

- 结构化形式化
- Lean 闭环验证
- 语义检索增强
- 项目级多人协作管理

Quokka：从长篇文献到可编译Lean项目



自然语言到形式化数学

- **核心目标**
  - 将长篇数学文献自动转化为可编译的 Lean 4 形式化证明
- **工作原理**
  - LLM 提案 × Lean 裁决
  - 文档级端到端架构
  - 文本片段级溯源保留
- **技术特点**
  - 输入：数学文献
  - 输出：可编译 Lean 项目
  - 自动分解 · 证明 · 验证

IMO 2026

Problem	Statement	Formalized	Solution	Lean	Lines	Time
Q1	Natural language	Lean statement	Markdown · PDF · HTML	Proof	481	50 min
Q2	Natural language	Lean statement	Markdown · PDF · HTML	Proof	772	88 min
Q3	Natural language	Lean statement	Markdown · PDF · HTML	Proof	1,724	105 min
Q4	Natural language	Lean statement	Markdown · PDF · HTML	Proof	673	70 min
Q5	Natural language	Lean statement	Markdown · PDF · HTML	Proof	370	46 min
Q6	Natural language	Lean statement	Markdown · PDF · HTML	Proof	440	43 min

Problem	stmt time	stmt tokens	proof time	proof tokens	Total time	Total tokens
P1	00:21:22	318,715	00:28:52	627,482	00:50:14	946,197
P2	00:14:41	296,165	01:12:54	1,376,600	01:27:34	1,672,765
P3	00:34:28	466,762	01:10:43	854,439	01:45:12	1,321,201
P4	00:36:43	706,044	00:33:15	415,329	01:09:58	1,121,373
P5	00:19:58	345,610	00:26:26	415,247	00:46:24	760,857
P6	00:16:17	334,493	00:26:41	499,169	00:42:58	833,662
Total	**02:23:29**	**2,467,789**	**04:18:52**	**4,188,266**	**06:42:21**	**6,656,055**

- **完成者：** 王梓琛
- **2026年7月18日**
- **工具链：Lean 4.32.0 / mathlib 4.32.0**
- **6 题的陈述与证明均已完成**
- **无任务或服务重试**
- **证明阶段占 Token 总量 62.93%**
- **P2 用量最高：1,672,765**
- **P3 耗时最长：01:45:12**

https://github.com/imathwy/IMO_test

教材形式化正在形成规模化知识资产

文档级自动形式化：从长篇数学文献生成可编译 Lean 项目，并沉淀为可复用知识资产

15 项目条目	11 本 书目	10,881 原文条目	约 3百万+行 Lean 代码	61,500 Lean 声明	53,059 / 86.3% 证明通过 / 声明进度

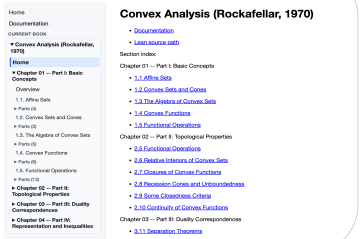
已形式化的知名书目 / 项目

- Tao《Analysis II》— 2.4k 声明 / 100%
- Rockafellar《Convex Analysis》— 7.3k 声明
- The Stacks Project — 13.9k 声明
- Nesterov《Convex Optimization》— 7.4k 声明
- Serre《Linear Representations》— 9.8k 声明
- Klenke《Probability Theory》— 6.0k 声明

Tao《Analysis II》(2022)



Rockafellar《Convex Analysis》(1970)



Verso 页面能力

把 Lean 项目发布成可浏览、可跳转、可溯源的教材索引

- 按原书章节自动生成导航
- Documentation / Lean 双入口
- 从网页条目回到 Lean 文件与原文 item
- 便于人工复查、复用与持续迭代

Quokka → ReasBook → ReasLib

https://github.com/optpku/ReasBook

自然语言与形式化语言推理协同研究

非线性规划猜想

- 问题：非线性等式约束、非凸优化

$$\min_{x\in\mathbb{R}^n} f(x) \qquad \text{s.t.} \qquad c(x)=0$$

固定罚参数 + 大步长乘子更新，
能否保持当时最优复杂度?

- 猜想与本文回答

$$\rho=\Theta(1):\quad \mathcal{O}(\varepsilon^{-3})\ ? \xRightarrow{\text{NR-LALM}} \mathcal{O}(\varepsilon^{-2})$$

Alacaoglu–Wright（AISTATS 2024）提出;
本文在 LALM 框架内给出肯定回答

Stephen J. Wright

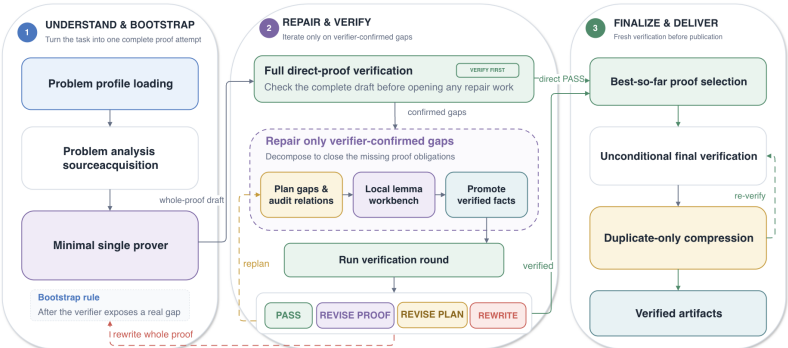


- 美国工程院院士
- SIAM Fellow
- George B. Dantzig奖
- ICM 2026 一小时报告人
- SIOPT、MPB 前主编
- NeurIPS Test of Time Award, 2020

自然语言智能体

构建多智能体系统，协同处理自然语言推理。

Proof Agent Architecture

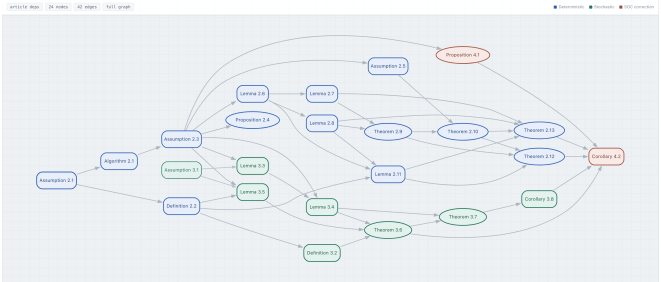


证明智能体从非形式化猜想出发，生成完整的自然语言证明草稿

从自然语言论证到 Lean 4 完整机器验证

交互式定理依赖图

公开网页原图：论文理论主链的 24 个节点、42 条直接依赖



蓝色 = 确定性 | 绿色 = 随机性 | 棕色 = 二阶修正（SOC）

在公开交互网页中，点击节点可查看 Lean 声明、源码位置和直接依赖

https://github.com/bqliu815/NR-LALM

论文主要结论已形式化

① **确定性**：固定参数达到目标精度的近似 KKT 点；每步仅需一次 SPD 线性系统求解。

$$N_{\mathrm{det}}(\varepsilon) = \mathcal{O}(\varepsilon^{-2})$$

② **随机性**：SPIDER 型估计配合 safeguarded restarts，得到匹配的期望复杂度。

$$\mathbb{E}[N_{\nabla f}] = \mathcal{O}(\varepsilon^{-3}), \qquad N_{c,\nabla c} = \mathcal{O}(\varepsilon^{-2})$$

③ **KL 收敛与二阶修正**：

$$\sum_{k=0}^{\infty} \left(\|x_{k+1} - x_k\| + \|\lambda_{k+1} - \lambda_k\| \right) < \infty$$

形式化规模

24	42	2,853	75,686	0
论文级节点	直接依赖	顶层声明	Lean 源码行	sorry / admit

◎ **已在 Lean 4 / mathlib v4.32.2 中完成理论主链机器验证；0 sorry，0 admit**

数学知识
从分散文献到可计算知识体系

数学知识百科全书

为什么数学需要百科全书

规模庞大
文献规模巨大，
无法逐篇梳理。

知识分散
定义、证明和算法分散在不同教材与论文中。

结构隐蔽
依赖路径与可复用证明模式难以识别。

面向智能体
大模型需要结构化对象，而不只是 PDF。

教材 / 论文
海量、分层、
适合人类阅读的资料

↓

类型化知识单元
定义、定理、引理、假设、算法与示例

↓

依赖图
关联、证明路径与
复用链接

↓

可检索知识底座
可索引、可查询、
可组合、可扩展的
知识层

百科全书项目带来的能力

统一本体
统一表示模型、目标与解概念。

结构化检索
可按定理目标、
假设条件或证明模式检索。

可复用证明
将引理、见证与
失败路径沉淀为可复用资源。

基准问题支撑
将开放问题转化为
可追溯、可分解的
研究目标。

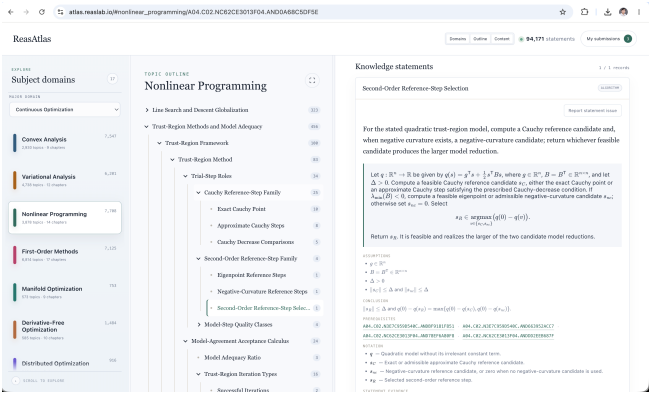
百科全书将教材级知识转化为可检索、可复用、可直接形式化的基础设施

优化百科全书

基于智能体的知识栈构建流程

1. **研究智能体**
   知识输入
   采集 · 筛选 · 溯源
2. **结构智能体**
   结构化知识设计
   定界 · 组织 · 扩展
3. **内容智能体**
   知识内容生成
   形式化 · 填充 · 关联
4. **审查智能体**
   验证与审查
   检查 · 验证 · 修订
5. **人工审查**
   纠错与改进
   修正 · 导出 · 更新

知识源层级



https://atlas.reaslab.io

ReasLab
智能数学推理引擎

面向数学研究一站式无缝工作流

AI 认知与协作环境
从工具调用到任务协同

AI + C++ Python … + Coq Lean … + Latex Markdown …

AI 在统一项目空间中理解问题目标、上下文与已有结果，协同调用计算、编译、验证和写作工具，推动**建模、求解、验证与报告**连续完成。


任务理解


工具调动


验证反馈


成果生成

“活”的云端协作生态
数学对象的“GitHub”

可追溯 **可复现** **可评审**

围绕模型、算法、证明、代码与报告，持续记录任务目标、符号约定、定义演化、实验结果和讨论反馈，让研究过程从一次性产出变成项目资产。


研究路径


过程记录


知识沉淀


自我演化

ReasLab 让数学知识被组织，研究过程被记录，AI 协作真正进入数学项目

最优化：简介

文再文

北京大学北京国际数学研究中心

教材《最优化：建模、算法与理论》配套电子教案

http://bicmr.pku.edu.cn/~wenzw/optbook.html

致谢：本教案由邓展望协助准备

提纲

1. 最优化问题概括
2. 实例：稀疏优化
3. 实例：低秩矩阵恢复
4. 实例：深度学习
5. 最优化基本概念

最优化问题的一般形式

最优化问题一般可以描述为

$$\begin{aligned} \min \quad & f(x), \\ \text{s.t.} \quad & x \in \mathcal{X}, \end{aligned} \tag{1}$$

- $x = (x_1, x_2, \cdots, x_n)^{\mathrm{T}} \in \mathbb{R}^n$是决策变量
- $f\colon \mathbb{R}^n \to \mathbb{R}$ 是目标函数
- $\mathcal{X} \subseteq \mathbb{R}^n$是约束集合或可行域，可行域包含的点称为可行解或可行点．记号 s.t. 是“subject to”的缩写，专指约束条件．
- 当$\mathcal{X} = \mathbb{R}^n$时，问题(1) 称为无约束优化问题．
- 集合$\mathcal{X}$通常可以由约束函数$c_i(x)\colon \mathbb{R}^n \to \mathbb{R}, i = 1, 2, \cdots, m + l$ 表达为如下具体形式：

$$\begin{aligned} \mathcal{X} = \{x \in \mathbb{R}^n \mid\ & c_i(x) \le 0, \quad i = 1, 2, \cdots, m, \\ & c_i(x) = 0, \quad i = m+1, m+2 \cdots, m+l\}. \end{aligned}$$

最优化问题的一般形式

- 在所有满足约束条件的决策变量中，使目标函数取最小值的变量x^* 称为优化问题(1)的最优解，即对任意$x \in \mathcal{X}$ 都有
$$f(x) \geq f(x^*).$$
- 如果求解在约束集合$\mathcal{X}$ 上目标函数$f(x)$的最大值，则问题(1) 的“min”应相应地替换为“max”.
- 注意到在集合$\mathcal{X}$ 上，函数f 的最小（最大）值不一定存在，但是其下（上）确界“inf f(sup f)” 总是存在的. 因此，当目标函数的最小(最大）值不存在时，我们便关心其下（上）确界，即将问题(1) 中的“min(max)” 改为“inf(sup)”.
- 为了叙述简便，问题(1) 中x为$\mathbb{R}^n$空间中的向量. 实际上，根据具体应用和需求，x还可以是矩阵、多维数组或张量等.

最优化问题的类型

最优化问题的具体形式非常丰富，可以按照目标函数、约束函数以及解的性质将其分类.

- 当目标函数和约束函数均为线性函数时，问题称为线性规划；
- 当目标函数和约束函数中至少有一个为非线性函数时，相应的问题称为非线性规划；
- 如果目标函数是二次函数而约束函数是线性函数则称为二次规划；
- 包含非光滑函数的问题称为非光滑优化；
- 不能直接求导数的问题称为无导数优化；
- 变量只能取整数的问题称为整数规划；
- 在线性约束下极小化关于半正定矩阵的线性函数的问题称为半定规划，其广义形式为锥规划.

最优化问题的类型

- 最优解只有少量非零元素的问题称为稀疏优化；
- 最优解是低秩矩阵的问题称为低秩矩阵优化.
- 此外还有几何优化、二次锥规划、张量优化、鲁棒优化、全局优化、组合优化、网络规划、随机优化、动态规划、带微分方程约束优化、微分流形约束优化、分布式优化等.
- 就具体应用而言，问题(1) 可涵盖统计学习、压缩感知、最优运输、信号处理、图像处理、机器学习、强化学习、模式识别、金融工程、电力系统等领域的优化模型.

最优化问题的应用

数学建模很容易给出应用问题不同的模型，可以对应性质很不相同的问题，其求解难度和需要的算法也将差别很大．在投资组合优化中，人们希望通过寻求最优的投资组合以降低风险、提高收益．

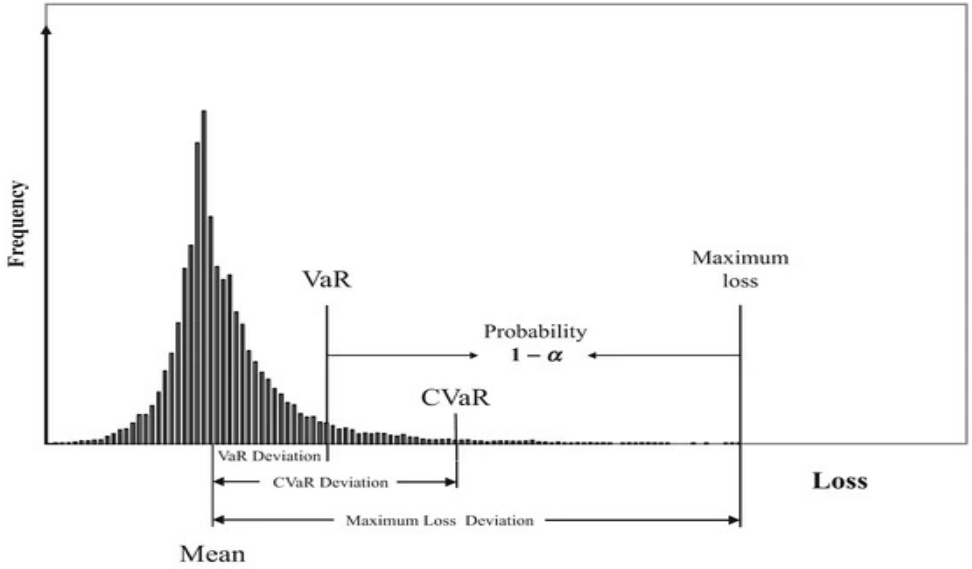
- 这时决策变量x_i表示在第i项资产上的投资额，向量$x \in \mathbb{R}^n$表示整体的投资分配．
- 约束条件可能为总资金数、每项资产的最大（最小）投资额、最低收益等．
- 目标函数通常是某种风险度量．
- 如果是极小化收益的方差，则该问题是典型的二次规划.
- 如果极小化风险价值(value at risk)函数，则该问题是混合整数规划
- 如果极小化条件风险价值（conditional value at risk)函数，则该问题是非光滑优化，也可以进一步化成线性规划．

投资组合优化

- r_i, 随机变量, 股票的回报率i
- x_i, 投资于股票的相对金额i
- 回报: $r = r_1x_1 + r_2x_2 + \ldots + r_nx_n$
- 期望回报: $R = E(r) = \sum E(r_i)x_i = \sum \mu_i x_i$
- 风险: $V = Var(r) = \sum_{i,j} \sigma_{ij}x_ix_j = x^\top \Sigma x$

$$\begin{aligned} \min\ & \frac{1}{2}x^\top \Sigma x, \\ \text{s.t.}\ & \sum \mu_i x_i \geq r_0 \\ & \sum x_i = 1, \\ & x_i \geq 0 \end{aligned} \qquad \begin{aligned} \min\ & \text{risk measure}, \\ \text{s.t.}\ & \sum \mu_i x_i \geq r_0 \\ & \sum x_i = 1, \\ & x_i \geq 0 \end{aligned}$$

风险度量


Frequency
VaR
Maximum loss
Probability
1 – α
CVaR
VaR Deviation
CVaR Deviation
Maximum Loss Deviation
Loss
Mean

提纲

1. 最优化问题概括
2. 实例：稀疏优化
3. 实例：低秩矩阵恢复
4. 实例：深度学习
5. 最优化基本概念

$$(\ell_0)\begin{cases}\min\limits_{x\in\mathbb{R}^n} & \|x\|_0,\\ \text{s.t.} & Ax=b.\end{cases}\quad(\ell_2)\begin{cases}\min\limits_{x\in\mathbb{R}^n} & \|x\|_2,\\ \text{s.t.} & Ax=b.\end{cases}\quad(\ell_1)\begin{cases}\min\limits_{x\in\mathbb{R}^n} & \|x\|_1,\\ \text{s.t.} & Ax=b.\end{cases}\tag{2}$$

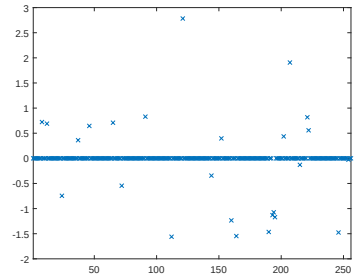
- 其中$\|x\|_0$是指x中非零元素的个数. 由于$\|x\|_0$是不连续的函数，且取值只可能是整数，ℓ_0问题实际上是NP难的，求解起来非常困难.
- 若定义ℓ_1范数：$\|x\|_1=\sum_{i=1}^n|x_i|$，则得到了另一个形式上非常相似的问题，又称$\ell_1$范数优化问题，基追踪问题
- ℓ_2范数：$\|x\|_2=\left(\sum_{i=1}^n x_i^2\right)^{1/2}$

稀疏优化

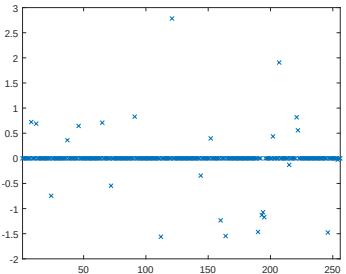
在MATLAB环境里构造A, u 和b：

```
m = 128; n = 256; A = randn(m, n);
u = sprandn(n, 1, 0.1); b = A * u;
```

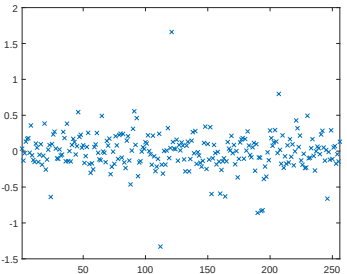
构造一个128×256矩阵A，它的每个元素都服从高斯（Gauss）随机分布．精确解u只有10%的元素非零，每一个非零元素也服从高斯分布



(a) 精确解u



(b) ℓ_1问题的解



(c) ℓ_2问题的解

Figure: 稀疏优化的例子

LASSO问题

考虑带ℓ_1范数正则项的优化问题

$$\min_{x\in\mathbb{R}^n} \quad \mu\|x\|_1 + \frac{1}{2}\|Ax-b\|_2^2, \tag{3}$$

其中$\mu > 0$是给定的正则化参数.

- 问题(3) 又称为LASSO（least absolute shrinkage and selection operator），该问题可以看成是问题(2) 的二次罚函数形式.
- 由于它是无约束优化问题，形式上看起来比问题(2) 简单. 课件关注的大部分数值算法都将针对问题(2) 或问题(3) 给出具体形式. 因此全面掌握它们的求解方法是掌握基本最优化算法的一个标志.

提纲

1. 最优化问题概括
2. 实例：稀疏优化
3. 实例：低秩矩阵恢复
4. 实例：深度学习
5. 最优化基本概念

低秩矩阵恢复

- 某视频网站提供了约48万用户对1万7千多部电影的上亿条评级数据，希望对用户的电影评级进行预测，从而改进用户电影推荐系统，为每个用户更有针对性地推荐影片.

- 显然每一个用户不可能看过所有的电影，每一部电影也不可能收集到全部用户的评级. 电影评级由用户打分1 星到5 星表示，记为取值1~5 的整数. 我们将电影评级放在一个矩阵M中，矩阵M的每一行表示不同用户，每一列表示不同电影. 由于用户只对看过的电影给出自己的评价，矩阵M中很多元素是未知的

	电影1	电影2	电影3	电影4	…	电影n
用户1	4	?	?	3	⋯	?
用户2	?	2	4	?	⋯	?
用户3	3	?	?	?	⋯	?
用户4	2	?	5	?	⋯	?
⋮	⋮	⋮	⋮	⋮		⋮
用户m	?	3	?	4	⋯	?

低秩矩阵恢复问题的性质

该问题在推荐系统、图像处理等方面有着广泛的应用。

- 由于用户对电影的偏好可进行分类，按年龄可分为：年轻人，中年人，老年人；且电影也能分为不同的题材：战争片，悬疑片，言情片等。故这类问题隐含的假设为补全后的矩阵应为低秩的。即矩阵的行与列会有“合作”的特性，故该问题具有别名“collaborative filtering”。
- 除此之外，由于低秩矩阵可分解为两个低秩矩阵的乘积，所以低秩限制下的矩阵补全问题是比较实用的，这样利于储存且有更好的诠释性。
- 有些用户的打分可能不为自身真实情况，对评分矩阵有影响，所以原矩阵是可能有噪声的。

低秩矩阵恢复

由上述分析可以引出该问题：

- 令Ω是矩阵M中所有已知评级元素的下标的集合，则该问题可以初步描述为构造一个矩阵X，使得在给定位置的元素等于已知评级元素，即满足$X_{ij} = M_{ij},\ (i,j) \in \Omega$.
- 低秩矩阵恢复（low rank matrix completion）

$$\begin{aligned} \min_{X \in \mathbb{R}^{m \times n}} \quad & \text{rank}(X), \\ \text{s.t.} \quad & X_{ij} = M_{ij},\ (i,j) \in \Omega. \end{aligned} \tag{4}$$

  $\text{rank}(X)$正好是矩阵X所有非零奇异值的个数

- 矩阵X的核范数（nuclear norm）为矩阵所有奇异值的和，即：$\|X\|_* = \sum_i \sigma_i(X)$:

$$\begin{aligned} \min_{X \in \mathbb{R}^{m \times n}} \quad & \|X\|_*, \\ \text{s.t.} \quad & X_{ij} = M_{ij},\ (i,j) \in \Omega. \end{aligned} \tag{5}$$

- 可以证明问题(5) 是一个凸优化问题，并且在一定条件下它与问题(4) 等价.
- 也可以将问题(5) 转换为一个半定规划问题，但是目前半定规划算法所能有效求解的问题规模限制了这种技术的实际应用.
- 考虑到观测可能出现误差，对于给定的参数$\mu > 0$，给出该问题的二次罚函数形式：

$$\min_{X \in \mathbb{R}^{m \times n}} \quad \mu \|X\|_* + \frac{1}{2} \sum_{(i,j) \in \Omega} (X_{ij} - M_{ij})^2. \tag{6}$$

低秩矩阵恢复

- 秩r情形：$X = LR^T$,其中$L \in \mathbb{R}^{m\times r}$,$R \in \mathbb{R}^{n\times r}$ 并且$r \ll \min(m,n)$.则可将问题写为

$$\min_{\boldsymbol{L},\boldsymbol{R}} \sum_{(i,j)\in\omega} \left(\left[\boldsymbol{L}\boldsymbol{R}^T\right]_{ij} - M_{ij} \right)^2 + \alpha \left\|\boldsymbol{L}\right\|_F^2 + \beta \left\|\boldsymbol{R}\right\|_F^2$$

- 在该问题中，矩阵X在定义中已为秩r矩阵，所以没有必要再加上秩约束正则项。α,β为正则化参数，这里正则化的作用是消除解L,R在放缩意义下的不唯一性.
- 此时L,R矩阵中的数字之和为$(m+n)r$,远小于np,不过此时问题是非凸的.
- 尽管这个该问题是非凸的，但在某种意义上它是一个可处理问题的近似：如果对X有一个完整的观察，那么秩-r近似可以通过X的奇异值分解来找到，并根据r导出的左奇异向量和右奇异向量定义L和R。

提纲

1. 最优化问题概括
2. 实例：稀疏优化
3. 实例：低秩矩阵恢复
4. 实例：深度学习
5. 最优化基本概念

机器学习中典型问题形式

很多机器学习中的问题可以写为：

$$\min_{x\in\mathcal{W}} \quad \sum_{i=1}^{N} \frac{1}{2}\|a_i^\top x - b_i\|_2^2 + \mu\varphi(x) \qquad \text{线性回归}$$

$$\min_{x\in\mathcal{W}} \quad \frac{1}{N}\sum_{i=1}^{N} \log(1+\exp(-b_i a_i^\top x)) + \mu\varphi(x) \qquad \text{逻辑回归}$$

$$\min_{x\in\mathcal{W}} \quad \frac{1}{N}\sum_{i=1}^{N} \ell(f(a_i,x),b_i) + \mu\varphi(x) \qquad \text{一般形式}$$

- (a_i, b_i) 是给定的数据对，b_i 是数据a_i对应的标签
- $\ell_i(\cdot)$: 度量模型拟合数据点i的程度(避免拟合不足)
- $\varphi(x)$: 避免过拟合的正则项: $\|x\|_2^2$ 或者$\|x\|_1$等等
- $f(a,x)$: 线性函数或者由深度神经网络构造的模型

上述过程可用下图表示:

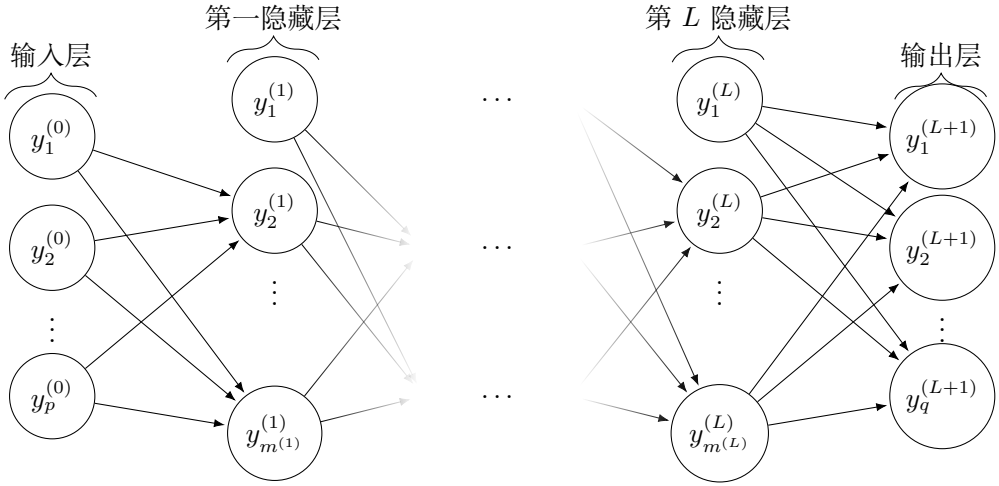


Figure: 带p个输入单元和q个输出单位的$(L+2)$层感知机的网络图。第l个隐藏层包含$m^{(l)}$个神经

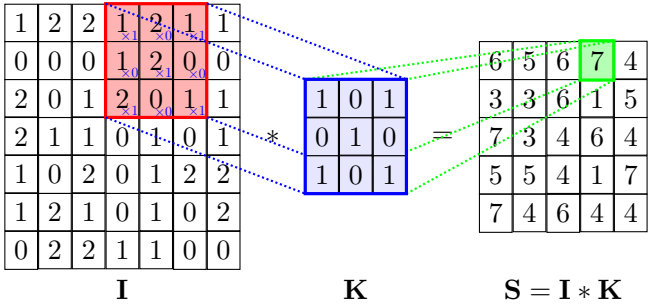
卷积神经网络Convolutional neural network (CNN)

- 给定二维图像$\boldsymbol{I} \in \mathbb{R}^{n\times n}$和卷积核$\boldsymbol{K} \in \mathbb{R}^{k\times k}$，定义卷积操作$\boldsymbol{S} = \boldsymbol{I} * \boldsymbol{K}$，它的元素是

$$\boldsymbol{S}_{i,j} = \langle \boldsymbol{I}(i : i+k-1, j : j+k-1), \boldsymbol{K} \rangle,$$

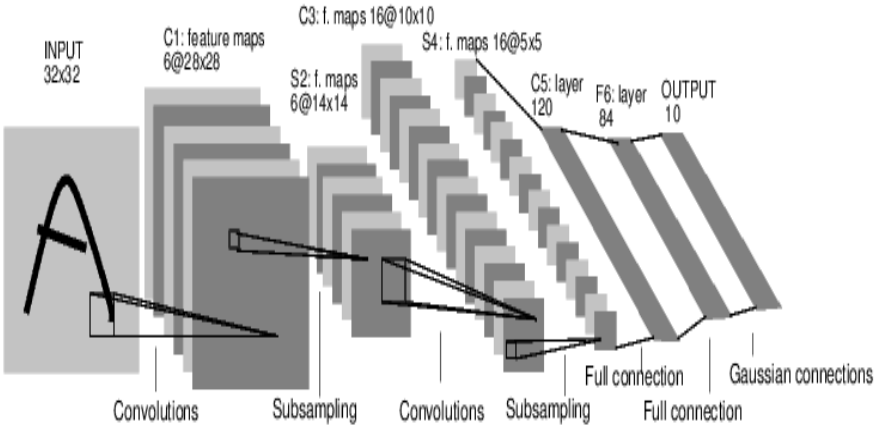
  其中两个矩阵X, Y的内积是它们相应元素乘积之和

- 生成的结果$\boldsymbol{S}$可以根据卷积核的维数、$\boldsymbol{I}$的边界是否填充、卷积操作时滑动的大小等相应变化.



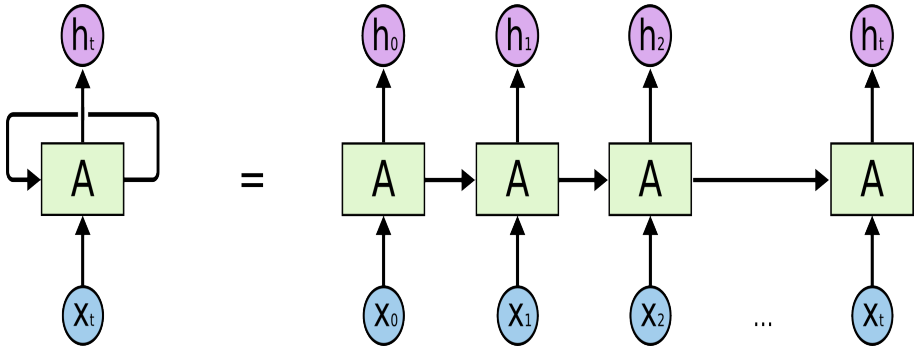
卷积神经网络Convolutional neural network (CNN)

LeCun等人开创性的建立了数字分类的神经网络。几家银行使用它来识别支票上的手写数字。



递归神经网络Recurrent neural networks (RNN)

RNN建立在与前馈神经网络相同的计算单元上。RNN不必分层组织，并且允许定向循环。这样一来，他们就可以拥有内部存储器，从而可以处理顺序数据。如图所示，可以将RNN按顺序“展开”，从而将RNN转换为常规前馈神经网络。



提纲

1. 最优化问题概括
2. 实例：稀疏优化
3. 实例：低秩矩阵恢复
4. 实例：深度学习
5. 最优化基本概念

全局和局部最优解

在求解最优化问题之前，先介绍最小化问题(1) 的最优解的定义.

定义 (最优解)

对于可行点$\bar{x}$ (即$\bar{x} \in \mathcal{X}$)，定义如下概念：

(1) 如果$f(\bar{x}) \leq f(x), \forall x \in \mathcal{X}$，那么称$\bar{x}$ 为问题(1) 的全局极小解（点），有时也称为（全局）最优解或最小值点；

(2) 如果存在$\bar{x}$ 的一个ε 邻域$N_\varepsilon(\bar{x})$ 使得$f(\bar{x}) \leq f(x), \forall x \in N_\varepsilon(\bar{x}) \cap \mathcal{X}$, 那么称$\bar{x}$ 为问题(1) 的局部极小解（点），有时也称为局部最优解；

(3) 进一步地，如果有$f(\bar{x}) < f(x), \forall x \in N_\varepsilon(\bar{x}) \cap \mathcal{X}$, 且$x \neq \bar{x}$ 成立，则称$\bar{x}$ 为问题(1) 的严格局部极小解（点）.

如果一个点是局部极小解，但不是严格局部极小解，则称为非严格局部极小解. 在图4中，我们以一个简单的函数为例，指出了其全局与局部极小解.

全局和局部最优解

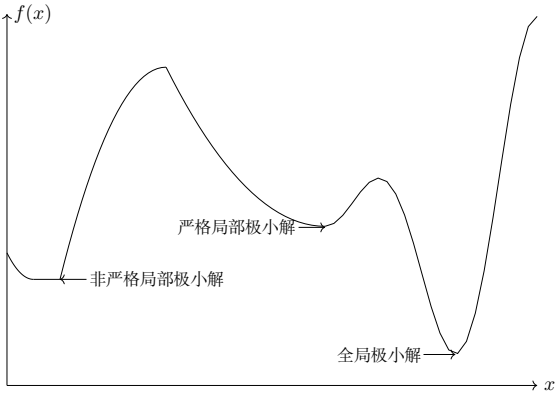


Figure: 函数的全局极小、严格局部极小和非严格局部极小解

在问题(1)的求解中，我们想要得到的是其全局最优解，但是由于实际问题的复杂性，往往只能够得到其局部最优解.

优化算法收敛性

由于实际问题往往是没有办法显式求解的，因此常采用迭代算法.

- 迭代算法的基本思想是：从一个初始点x^0出发，按照某种给定的规则进行迭代，得到一个序列$\{x^k\}$. 如果迭代在有限步内终止，那么最后一个点就是优化问题的解. 如果迭代点列是无穷集合，那么希望该序列的极限点（或者聚点）则为优化问题的解.
- 在算法设计中，还需考虑算法产生的点列是否收敛到优化问题的解. 给定初始点x^0，记算法迭代产生的点列为$\{x^k\}$. 如果$\{x^k\}$在某种范数$\|\cdot\|$的意义下满足$\lim_{k\to\infty}\|x^k - x^*\| = 0$,且收敛的点$x^*$为一个局部（全局）极小解，那么我们称该点列收敛到局部（全局）极小解，相应的算法称为是依点列收敛到局部（全局）极小解的.

优化算法收敛性

- 如果从任意初始点x^0出发，算法都是依点列收敛到局部（全局）极小解的，我们称该算法是全局依点列收敛到局部（全局）极小解的．记对应的函数值序列$\{f(x^k)\}$，可以定义算法的（全局）依函数值收敛到局部（全局）极小值的概念．
- 对于凸优化问题，因为其任何局部最优解都为全局最优解，算法的收敛性都是相对于其全局极小而言的．
- 除了点列和函数值的收敛外，实际中常用的还有每个迭代点的最优性条件（如无约束优化问题中的梯度范数，约束优化问题中的最优性条件违反度等等）的收敛．
- 对于带约束的情形，给定初始点x^0，算法产生的点列$\{x^k\}$不一定是可行的（即$x^k \in \mathcal{X}$未必对任意k成立）．考虑到约束违反的情形，我们需要保证$\{x^k\}$在收敛到x^*的时候，其违反度是可接受的．除此要求之外，算法的收敛性的定义和无约束情形相同．

算法的渐进收敛速度

设$\{x^k\}$为算法产生的迭代点列且收敛于x^*

- 算法（点列）**Q-**线性收敛: 对充分大的k有

$$\frac{\|x^{k+1}-x^*\|}{\|x^k-x^*\|} \le a, \quad a \in (0,1)$$

- 算法（点列）**Q-**超线性收敛: 对充分大的k有

$$\lim_{k\to\infty} \frac{\|x^{k+1}-x^*\|}{\|x^k-x^*\|} = 0,$$

- 算法（点列）**Q-**次线性收敛: 对充分大的k有

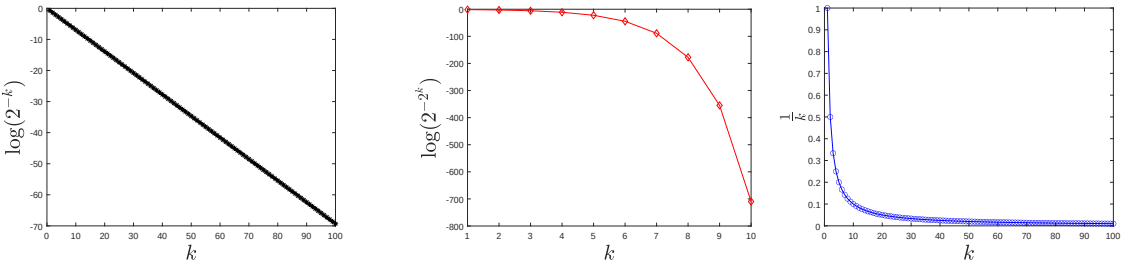
$$\lim_{k\to\infty} \frac{\|x^{k+1}-x^*\|}{\|x^k-x^*\|} = 1,$$

- 算法（点列）**Q-**二次收敛: 对充分大的k有

$$\frac{\|x^{k+1}-x^*\|}{\|x^k-x^*\|^2} \le a, \quad a > 0,$$

算法的渐进收敛速度

我们举例来更直观地展示不同的Q-收敛速度，参见下图（图中对所考虑的点列作了适当的变换）．点列$\{2^{-k}\}$是Q-线性收敛的，点列$\{2^{-2^k}\}$是Q-二次收敛的（也是Q-超线性收敛的），点列$\{\frac{1}{k}\}$是Q-次线性收敛的．一般来说，具有Q-超线性收敛速度和Q-二次收敛速度的算法是收敛较快的



(a) Q-线性收敛　(b) Q-二次收敛　(c) Q-次线性收敛

Figure: 不同Q-收敛速度比较

算法的渐进收敛速度

- 算法（点列）**R-**线性收敛: 设$\{x^k\}$为算法产生的迭代点且收敛于x^*，若存在Q-线性收敛于0 的非负序列t_k 并且

$$\|x^k - x^*\| \leq t_k$$

对任意的k成立．类似地，可定义**R-**超线性收敛和**R-**二次收敛等收敛速度．从R-收敛速度的定义可以看出序列$\{\|x^k - x^*\|\}$ 被另一趋于0 的序列$\{t_k\}$控制．当知道t_k的形式时，我们也称算法（点列）的收敛速度为$\mathcal{O}(t_k)$.

- 算法复杂度．设x^*为全局极小点，某一算法产生的迭代序列$\{x^k\}$满足

$$f(x^k) - f(x^*) \leq \frac{c}{\sqrt{k}}, \quad \forall\, k > 0,$$

其中$c > 0$为常数。如果需要计算算法满足精度$f(x^k) - f(x^*) \leq \varepsilon$所需的迭代次数，只需令$\frac{c}{\sqrt{k}} \leq \varepsilon$ 则得到$k \geq \frac{c^2}{\varepsilon^2}$，因此该优化算法对应的（迭代次数）复杂度为$N(\varepsilon) = \mathcal{O}\left(\frac{1}{\varepsilon^2}\right)$.

优化算法的收敛准则

为了使算法能在有限步内终止，一般会通过一些收敛准则来保证迭代停在问题的一定精度逼近解上.

- 对于无约束优化问题，常用的收敛准则有

$$\frac{f(x^k)-f^*}{\max\{|f^*|,1\}} \le \varepsilon_1, \quad \|\nabla f(x^k)\| \le \varepsilon_2, \tag{7}$$

其中$\varepsilon_1, \varepsilon_2$ 为给定的很小的正数，$\|\cdot\|$ 表示某种范数（这里可以简单理解为ℓ_2范数：$\|x\|_2 = \left(\sum_{i=1}^n x_i^2\right)^{1/2}$，$f^*$ 为函数f 的最小值以及$\nabla f(x^k)$ 表示函数f在点x 处的梯度.

- 对于约束优化问题，还需要考虑约束违反度. 也即要求最后得到的点满足

$$c_i(x^k) \le \varepsilon_3, \ i = 1, 2, \cdots, m,$$
$$|c_i(x^k)| \le \varepsilon_4, \ i = m+1, m+2, \cdots, m+l,$$

其中$\varepsilon_3, \varepsilon_4$ 为很小的正数，用来刻画x^k 的可行性.

优化算法的收敛准则

- 除了约束违反度之外，也要考虑x^k与最优解之间的距离，如(7)式中给出的函数值与最优值的相对误差. 由于一般情况下事先并不知道最优解，在最优解唯一的情形下一般使用某种基准算法来得到x^*的一个估计，之后计算其与x^k的距离以评价算法的性能.
- 因为约束的存在，不能简单地用目标函数的梯度来判断最优性，实际中采用的判别准则是点的最优性条件的违反度.
- 对于具体的算法，根据其设计的出发点，不一定能得到一个高精度的逼近解. 为了避免无用的计算开销，需要一些停机准则来及时停止算法的进行. 常用的停机准则有

$$\frac{\|x^{k+1}-x^k\|}{\max\{\|x^k\|,1\}} \le \varepsilon_5, \quad \frac{|f(x^{k+1})-f(x^k)|}{\max\{|f(x^k)|,1\}} \le \varepsilon_6,$$

  其分别表示相邻迭代点和其对应目标函数值的相对误差很小.
- 在算法设计中，这两个条件往往只能反映迭代点列接近收敛，但不能代表收敛到优化问题的最优解.